BUSRADAR-MS — A Real-Time Bus Tracking Pipeline

A self-directed learning project building an end-to-end streaming data pipeline from scratch.

status actively developed | live busradar-ms.live | platform Databricks

transformation dbt | streaming Apache Kafka

🔗 **Live dashboard: busradar-ms.live**

Project Overview

Busradar is a real-time data pipeline that ingests live public transit positions from all buses in Münster, Germany (my hometown). A single Kafka topic acts as the central event backbone, feeding two independent downstream consumers that each serve a different purpose:

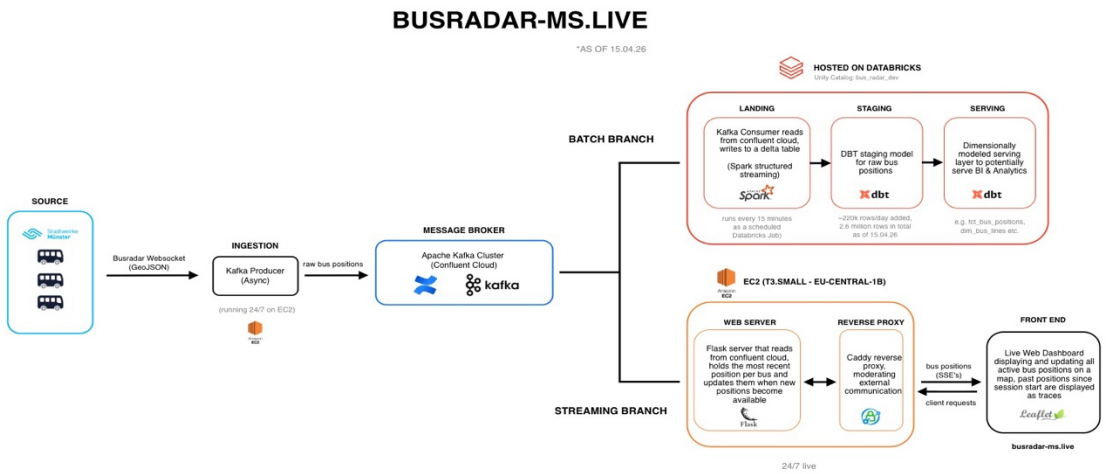
- **Batch branch:** Kafka → Spark Structured Streaming → Delta Lake → dbt , building a structured analytical layer for historical analysis.
- **Streaming branch:** Kafka → Flask (background consumer) → Server-Sent Events → Leaflet.js, powering a live map dashboard that visualizes every active bus in real time.

The data source is the real-time WebSocket feed from Stadtwerke Münster (the city's public transit operator), which pushes GeoJSON-encoded bus positions every few seconds. This dataset reflects actual vehicle movements across 40+ bus lines, with operational metadata including delays, stop sequences, and route assignments.

The project serves as a hands-on learning path through the modern data stack. Every component — from the async WebSocket consumer to the dbt serving layer to the EC2 deployment — was built incrementally to develop practical fluency with production-grade tools. The live deployment at busradar-ms.live provides a tangible, running proof of the full system.

Architecture

The pipeline uses a streaming-first architecture with a shared Kafka backbone. Both branches consume independently from the same topic, but they serve fundamentally different purposes — the batch branch builds a durable analytical store (~220k rows/day, 2.6M+ rows accumulated), while the streaming branch powers a live operational dashboard. This is not a lambda architecture (where both branches compute the same result and a serving layer merges them), but rather two independent consumers each solving a different problem from the same event stream.



Data flow:

1. **Ingestion** — A Python WebSocket client connects to Stadtwerke Münster's live feed, flattens GeoJSON features into individual bus position events, and produces them to Kafka using `fahrtbezeichner` (trip identifier) as the message key.

2. **Batch branch** — Spark Structured Streaming on Databricks consumes from Kafka using an `AvailableNow` trigger on a 15-minute schedule (configured as a Databricks Job). Raw positions land in a Delta Lake bronze table under Unity Catalog. dbt then transforms bronze into a staging view and serving-layer materialized views (`fct_bus_positions`, `dim_bus_lines`, `dim_dates`) for analytical queries via Databricks SQL.

3. **Streaming branch** — A Flask web server runs a background Kafka consumer thread that maintains an in-memory dictionary of the latest position per bus. An SSE endpoint (`/stream`) snapshots this state every second, evicts stale buses, and pushes JSON to all connected browser clients. The frontend renders positions on a Leaflet.js map with direction arrows, movement trails, and color-coding.

Tech Stack

Layer	Technology	Purpose
Data Source	Stadtwerke Münster WebSocket	Real-time GeoJSON bus positions
Ingestion	Python, `websockets`, `confluent-kafka`	Async WebSocket consumer, Kafka producer
Message Broker	Apache Kafka (Confluent Cloud)	Central event backbone, 6 partitions
Batch Processing	Spark Structured Streaming (Databricks)	Kafka → Delta Lake micro-batch ingestion
Storage	Delta Lake, Unity Catalog	Bronze table, schema enforcement, checkpoints
Transformation	dbt (dbt-databricks)	Staging views, serving materialized views, tests
Analytics	Databricks SQL	Query layer on dbt serving models
Web Server	Flask, Server-Sent Events	Real-time API with background Kafka consumer
Frontend	Leaflet.js, HTML/CSS/JS	Live map with trails, popups, line filtering
Infrastructure	AWS EC2 (t3.small), Caddy, systemd	Hosting, reverse proxy (auto-HTTPS), process management
Domain	Strato	`busradar-ms.live`

Repository Structure

```
busradar/
├── kafka/                          # Ingestion layer
│   ├── producer.py                 # WebSocket → Kafka producer (async)
│   ├── consumer.py                 # Standalone test consumer (debugging)
│   ├── config.py                   # Confluent Cloud connection config
│   ├── databricks_consumer.ipynb   # Spark Structured Streaming consumer
│   ├── requirements.txt
│   └── .env.example                # Required environment variables
│
├── web/                            # Streaming branch — live dashboard
│   ├── app.py                      # Flask SSE server + background Kafka con
sumer
│   ├── templates/
│   │   └── index.html              # Leaflet.js map with full UI
│   └── requirements.txt
│
├── dbt_bus_radar_ms/               # Transformation layer (dbt project)
│   ├── dbt_project.yml             # Project config, schema routing
│   ├── models/
│   │   ├── staging/
│   │   │   ├── stg_raw_bus_positions.sql   # View on bronze table
│   │   │   └── _src_bus_positions.yml      # Source definition
│   │   └── serving/
│   │       ├── fct_bus_positions.sql       # Fact table (materialized view)
│   │       ├── dim_bus_lines.sql           # Line dimension (materialized vi
ew)
│   │       └── dim_dates.sql              # Date dimension (table, dbt_date)
│   ├── macros/
│   │   └── generate_schema_name.sql       # Schema routing override
│   ├── packages.yml                       # dbt_utils, dbt_date
│   └── seeds/                             # Planned: dim_stations
│
├── docs/                           # Documentation assets
│   └── busradar-ms-architecture.svg  # Pipeline architecture diagram
│
├── .gitignore
└── README.md
```

Pipeline Deep-Dive

Ingestion: WebSocket → Kafka

The producer (`kafka/producer.py`) establishes a persistent WebSocket connection to `wss://websocket.busradar.conterra.de`. Each incoming message is a GeoJSON `FeatureCollection` containing the current state of all active buses.

The producer flattens each feature into an individual bus position event, handling two operation types:

- **`MODIFY`** — A bus has moved. The event includes coordinates, line information, delay, stop sequence, and operational timestamps. Latitude and longitude are extracted from GeoJSON geometry coordinates.
- **`REMOVE`** — A bus has gone out of service. The event carries only identifiers, no coordinates.

Each event is produced to the `bus_locations_ms_raw` topic with `fahrtbezeichner` (the unique trip identifier) as the Kafka message key. This ensures all events for a given trip land on the same partition, preserving ordering per bus. The producer handles `BufferError` with backpressure (poll-and-retry) and uses asynchronous delivery callbacks for error tracking.

On WebSocket disconnection, the `websockets` library's built-in reconnection (`async for websocket in websockets.connect(uri)`) automatically re-establishes the connection.

Batch Branch: Kafka → Delta Lake → dbt

Spark Structured Streaming (`kafka/databricks_consumer.ipynb`) reads from the Kafka topic on Databricks. It uses the `AvailableNow` trigger — processing all available messages in a single micro-batch, then terminating. A Databricks Job runs this notebook on a 15-minute cron schedule, giving near-real-time analytical freshness without the cost of a continuously running cluster.

The raw JSON payloads are parsed against a defined `StructType` schema, enriched with `kafka_timestamp` and a `_loaded_at` audit column, and appended to the `bus_radar_dev.bronze.raw_positions` Delta table. Checkpoints are stored in a Unity Catalog Volume to ensure exactly-once processing semantics across job runs.

dbt (`dbt_bus_radar_ms/`) then transforms the bronze layer into analytics-ready models:

Model	Type	Schema	Description
`stg_raw_bus_positions`	View	`bronze`	Pass-through staging view on the raw Delta table
`fct_bus_positions`	Materialized View	`analytics`	Fact table with parsed timestamps, delay, hour/minute extraction, bus line key
`dim_bus_lines`	Materialized View	`analytics`	Distinct bus lines with route direction (deduped from raw)
`dim_dates`	Table	`analytics`	Standard date dimension (1990–2050, via `dbt_date`)

The project uses a `generate_schema_name` macro override to route models directly to their target schemas (`bronze`, `analytics`) rather than prepending the dbt target schema — a deliberate design choice for cleaner Unity Catalog organization.

Materialized views auto-refresh on query in Databricks SQL, so the serving layer stays current without manual orchestration.

Streaming Branch: Kafka → Flask SSE → Live Map

The Flask application (`web/app.py`) runs a background daemon thread with a dedicated Kafka consumer. This consumer polls the same `bus_locations_ms_raw` topic and maintains a shared in-memory dictionary (`bus_positions`) keyed by `fahrtbezeichner`:

- **`MODIFY`** events upsert the dictionary entry with the latest position and a `_last_seen` timestamp.
- **`REMOVE`** events delete the entry.

The `/stream` SSE endpoint snapshots this dictionary every second, evicts entries older than the staleness threshold (currently 10 minutes), serializes to JSON, and pushes to all connected browser clients via Server-Sent Events.

The frontend (`web/templates/index.html`) connects to `/stream` using the browser's native `EventSource` API and renders each bus as a color-coded marker on a Leaflet.js map. Features include direction-of-travel arrows (calculated from successive positions), fading movement trails, delay color-coding, per-line filtering, and a searchable sidebar with live statistics.

> **Transparency note:** The frontend (HTML/CSS/JavaScript) was built with AI assistance. The backend pipeline — from WebSocket ingestion through Kafka, Spark, Delta Lake, dbt, Flask, and EC2 deployment — is the author's own work.

Infrastructure & Deployment

The streaming branch runs on an **AWS EC2 t3.small** instance (`eu-central-1b`). The deployment uses:

- **systemd** — Process management for the Kafka producer and Flask web server. Unit files ensure automatic restart on failure and structured logging via `journald`.
- **Caddy** — Reverse proxy with automatic HTTPS certificate provisioning via Let's Encrypt. Serves the Flask app on port 443 and handles TLS termination.
- **Domain** — `busradar-ms.live` registered via Strato (€0.25/month), DNS A-record pointed to the EC2 Elastic IP.

The Databricks-side batch processing runs on serverless compute, triggered by a Databricks Job on a 15-minute cron schedule.

Roadmap

This project is under active development. Planned improvements include:

Near-term - [ ] Prod / dev EC2 instance split — dedicated production instance with DNS repointing, current instance becomes dev - [ ] GTFS-based route snapping — project GPS positions onto official Stadtwerke Münster route shapes using GTFS static feed data - [ ] `dim_stations` seed — enrich the dbt layer with a station/stop dimension from GTFS stops data - [ ] Staleness threshold tuning — optimize the eviction window based on observed bus reporting intervals - [ ] Cron overnight shutdown (1–5am) — reduce EC2 costs during no-service hours

Medium-term - [ ] Self-hosted Kafka (KRaft) — single-broker setup on a second EC2 instance as a learning exercise, replacing Confluent Cloud - [ ] Producer heartbeat logging — periodic health-check log lines for monitoring producer liveness

Future - [ ] Frontend event tracking → Kafka → Databricks — a second source system feeding the same analytical pipeline, demonstrating multi-source ingestion - [ ] Log offloading to CloudWatch — structured log pipeline for operational visibility - [ ] Silver snapped-positions table — store GTFS-matched positions as a curated analytical layer

---

Getting Started

Prerequisites

- Python 3.11+
- A Confluent Cloud account (or self-hosted Kafka cluster)
- Databricks workspace with Unity Catalog enabled
- AWS EC2 instance (for deployment) or local machine (for development)

Setup

1. **Clone the repository**

   ```
   git clone https://github.com/<your-username>/busradar.git
   cd busradar
   ```

2. **Configure environment variables**

   ```
   cp kafka/.env.example kafka/.env
   # Edit kafka/.env with your Confluent Cloud credentials
   ```

   Required variables:

   ```
   CONFLUENT_BOOTSTRAP_SERVERS=<your-cluster-url>
   CONFLUENT_SASL_SSL=SASL_SSL
   CONFLUENT_CONNECTION_METHOD=PLAIN
   CONFLUENT_USERNAME=<your-api-key>
   CONFLUENT_API_KEY=<your-api-secret>
   ```

3. **Install dependencies**

   ```
   # Kafka producer
   cd kafka && pip install -r requirements.txt

   # Web dashboard
   cd ../web && pip install -r requirements.txt
   ```

4. **Start the producer** (ingests live bus data into Kafka)

   ```
   cd kafka
   python producer.py
   ```

5. **Start the web dashboard** (consumes from Kafka, serves live map)

   ```
   cd web
   python app.py
   # Open http://localhost:8050 in your browser
   ```

6. **Run dbt models** (requires Databricks connection configured in `~/.dbt/profiles.yml`)

```
cd dbt_bus_radar_ms
dbt deps
dbt run
```

License

This project is for educational and portfolio purposes.

Built by **Lennart Rosenthal** as a self-directed data engineering learning project. busradar-ms.live